



SMART CONTRACT SECURITY REVIEW

PasswordStore Audit Report

PasswordStore

Version 1.0

Joev

Independent Smart Contract Security Researcher

May 31, 2026

Table of Contents

About the Auditor	2
Disclaimer	2
Protocol Summary	2
Audit Details	2
Scope	2
Roles	2
Methodology & Tools	2
Risk Classification	3
Severity Definitions	3
Executive Summary	3
Issues found	4
Findings Summary	4
Findings	4
Critical	4
High	4
[H-1] Storing the password on-chain makes it visible to anyone and no longer private	4
[H-2] TITLE <code>PasswordStore::setPassword</code> has no access controls, meaning a non-owner could change the password	5
Medium	6
Low	6
Informational	6
[I-1] <code>PasswordStore::getPassword</code> natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect.	6
Gas	6

About the Auditor

Joev is an independent smart contract security researcher.

- GitHub: <https://github.com/joevly>

Disclaimer

This document records a time-boxed security review performed by Joev on the specific code and commit identified in *Audit Details*. It describes the issues found within that window and is **not** a guarantee that the code is free of vulnerabilities — a clean report lowers risk, it does not remove it.

The review covers only the on-chain Solidity implementation in scope. Off-chain components, economic and incentive design, governance, deployment configuration, and third-party dependencies are out of scope unless explicitly stated. This review is not an endorsement of the protocol, its team, or its business, and nothing here is financial, investment, or legal advice. Ongoing testing, monitoring, and independent review remain the responsibility of the protocol team.

Protocol Summary

PasswordStore is a protocol used to store and retrieve user passwords. It's designed for single-user use, not multi-user use. Only the owner can set and access passwords, and non-owners cannot.

Audit Details

The findings in this document correspond to the following commit hash:

```
2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
```

Scope

```
./src/  
#-- PasswordStore.sol
```

- **Solc version:** 0.8.18
- **Chain(s) to deploy to:** Ethereum Mainnet.

Roles

- **Owner:** The user who can set the password and read the password.
- **Outsiders:** No one else should be able to set or read the password.

Methodology & Tools

This was a focused manual review of a single, small contract:

- **Manual review** — line-by-line reading of `PasswordStore.sol`, focused on the protocol's data privacy assumptions and access control.
- **On-chain storage inspection** — using `cast storage` against a local Anvil deployment to demonstrate that "private" on-chain state is publicly readable.

Given the size and simplicity of the contract, no automated static-analysis or fuzzing tooling was required.

Risk Classification

Severity is a function of **likelihood** (how easily the issue can be triggered) and **impact** (how damaging it is if triggered):

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

This Critical / High / Medium / Low model is the de-facto standard among audit firms and bug-bounty platforms (e.g. Cantina, OpenZeppelin, Immunefi). Informational and Gas findings sit outside the matrix as code-quality and optimization notes.

Severity Definitions

- **Critical** — directly and easily exploitable loss or theft of funds, or full protocol compromise. Must be fixed before deployment.
- **High** — loss of funds or a broken core invariant under realistic conditions; urgent fix.
- **Medium** — conditional or bounded harm, or protocol disruption where recovery is likely; fix recommended.
- **Low** — limited impact, unlikely preconditions, or a deviation from best practice with minor effect.
- **Informational** — no direct security impact: code quality, readability, unused code, events.
- **Gas** — optimizations that reduce gas with no change to behavior.

Executive Summary

As for the audit process, it didn't take much time because the protocol is simple and I didn't use any tools. In short, I found 3 vulnerabilities in the PasswordStore protocol, with 2 vulnerabilities rated HIGH and 1 vulnerabilities rated INFORMATIONAL. You can see and read the details below.

Issues found

Severity	Number of issues found
Critical	0
High	2
Medium	0
Low	0
Info	1
Gas	0
Total	3

Findings Summary

ID	Short title	Severity	Status
H-1	On-chain password is publicly readable	High	Open
H-2	<code>setPassword</code> has no access control	High	Open
I-1	<code>getPassword</code> natspec references missing param	Info	Open

Findings

Critical

No critical severity issues found.

High

HIGH [H-1] Storing the password on-chain makes it visible to anyone and no longer private

Description: All data stored on-chain is visible to anyone and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be a private variable and only accessed through the `PasswordStore::getPassword` function, which is intended to be only called by the owner of the contract.

We show one such method of reading any data off chain below.

Impact: Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept: (Proof of code)

The below test case shows how anyone can read the password directly from the blockchain.

1. Create a locally running chain

```
make anvil
```

2. Deploy the contract to the chain

```
make deploy
```

3. Run the storage tool We use 1 because that's the storage slot of `s_password` in the contract.

```
cast storage <CONTRACT_ADDRESS> 1 --rpc-url http://127.0.0.1:8545
```

You'll get an output that looks like this:

```
0x6d79506173737766f726400000000000000000000000000000000000000000014
```

You can then parse that hex to a string with:

```
cast parse-bytes32-string
0x6d79506173737766f7264000000000000000000000000000000000000000014
```

And get an output of:

```
myPassword
```

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain and then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the password. However, you'd also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

HIGH [H-2] TITLE `PasswordStore::setPassword` has no access controls, meaning a non-owner could change the password

Description: The `PasswordStore::setPassword` function is set to be an `external` function, however, that natspec of the function and overall purpose of the smart contract is that `This function allows only the owner to set a new password.`

```
function setPassword(string memory newPassword) external {
  @> // @audit - There are no access controls
    s_password = newPassword;
    emit SetNetPassword();
}
```

Impact: Anyone can set/change the password of the contract, severely breaking the contract intended functionality.

Proof of Concept: Add the following to the `PasswordStore.t.sol` test file.

```
function test_anyone_can_set_password(address randomAddress) public {
  vm.assume(randomAddress != owner);
  vm.prank(randomAddress);
  string memory expectedPassword = "myNewPassword";
  passwordStore.setPassword(expectedPassword);

  vm.prank(owner);
  string memory actualPassword = passwordStore.getPassword();
  assertEq(actualPassword, expectedPassword);
}
```

Recommended Mitigation: Add an access control conditional to the `setPassword` function.

```
if(msg.sender != s_owner) {
  revert PasswordStore__NotOwner();
}
```

Medium

No medium severity issues found.

Low

No low severity issues found.

Informational

INFO **[I-1]** `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect.

Description:

```
/*
 * @notice This allows only the owner to retrieve the password.
 * @param newPassword The new password to set.
 */
function getPassword() external view returns (string memory) {
```

The `PasswordStore::getPassword` function signature is `getPassword()` while the natspec say it should be `getPassword(string)`.

Impact: The natspec is incorrect.

Recommended Mitigation: Remove the incorrect natspec line.

```
- * @param newPassword The new password to set.
```

Gas

No gas optimizations reported.